

CYBER SECURITY iSPACE COLLEGE
Faculty of Information Technology

SESSION 2

C# Collections:
List, ArrayList & LinkedList

Data Structures and Algorithms — Lab 02

Course	Data Structures and Algorithms
Session	Session 02 — Collections: List, ArrayList, LinkedList
Language	C# (.NET 8 — Console Application)
IDE	Visual Studio 2022 / Visual Studio Code + .NET SDK
Duration	3 periods (≈ 135 minutes)
Prerequisites	Session 01: Algorithms & Complexity (completed)
Submission	https://drive.google.com/drive/folders/1L_cT8Zi2EPv8tIXrTNn7-04tVWSq_uHv?usp=sharing
Folder naming	StudentID_FullName (e.g. SE123456_NguyenVanA)

Session 2 — C# Collections: List, ArrayList & LinkedList

Part 1 — Introduction & Step-by-Step Examples | ≈ 45 min

1.1 Overview — Why Do We Need Collections?

In real-world programming, we rarely know in advance how many elements we need to store. A regular array (`int[]`) has a fixed size — create it with 10 slots and you can only store 10 elements. Collections solve this by automatically resizing as needed.

Three List Types in C# .NET

- ① `List<T>` — Generic, strongly-typed, high performance, most commonly used
- ② `ArrayList` — Non-generic (object), legacy from .NET 1.x, still supported
- ③ `LinkedList<T>` — Doubly-linked list, optimised for insert/delete in the middle

Required namespaces:

```
using System.Collections;    // cho ArrayList
```

```
using System.Collections.Generic; // cho List<T>, LinkedList<T>
```

Quick Comparison — Three List Types

Criterion	<code>List<T></code>	<code>ArrayList</code>	<code>LinkedList<T></code>
Type safety	✓ Strongly typed	✗ object	✓ Strongly typed
Index access [i]	✓ $O(1)$	✓ $O(1)$	✗ $O(n)$
Add to end (Add)	✓ $O(1)^*$	✓ $O(1)^*$	✓ $O(1)$
Insert/Remove at head/middle	✗ $O(n)$	✗ $O(n)$	✓ $O(1)$
Memory overhead	Low	High (boxing)	High (node pointers)
When to use?	Default — most use cases	Working with legacy code	Frequent insert/delete in middle

Note

* $O(1)$ amortized — internal array is occasionally resized, but average cost remains $O(1)$.

1.2 `List<T>` — Generic List

`List<T>` is the most widely used collection class in C#. It behaves like an automatically resizing array — internally it uses an array, but grows automatically when more space is needed.

Key Methods and Properties of List<T>

Method / Property	Description	Return type
<code>Add(T item)</code>	Append item to the end of the list	void
<code>AddRange(IEnumerable<T>)</code>	Append a collection to the end	void
<code>Insert(int i, T item)</code>	Insert item at position index i	void
<code>InsertRange(int i, col)</code>	Insert multiple items at position i	void
<code>Remove(T item)</code>	Remove the first occurrence of item	bool
<code>RemoveAt(int i)</code>	Remove element at index i	void
<code>RemoveRange(int i, n)</code>	Remove n elements starting at index i	void
<code>RemoveAll(Predicate<T>)</code>	Remove all elements matching condition	int
<code>Clear()</code>	Remove all elements	void
<code>Contains(T item)</code>	Check if item exists in the list	bool
<code>IndexOf(T item)</code>	Find first index of item (-1 if not found)	int
<code>LastIndexOf(T item)</code>	Find last index of item	int
<code>Find(Predicate<T>)</code>	Find first element matching predicate	T
<code>FindAll(Predicate<T>)</code>	Find all elements matching predicate	List<T>
<code>FindIndex(Predicate<T>)</code>	Find index of first matching element	int
<code>Sort()</code>	Sort ascending (default comparator)	void
<code>Sort(Comparison<T>)</code>	Sort with a custom comparison function	void
<code>Reverse()</code>	Reverse the order of elements	void
<code>ToArray()</code>	Convert to array T[]	T[]
<code>CopyTo(T[])</code>	Copy elements into an existing array	void
<code>ForEach(Action<T>)</code>	Iterate over each element	void
<code>Exists(Predicate<T>)</code>	Check if any element matches predicate	bool
<code>TrueForAll(Predicate)</code>	Check if ALL elements match predicate	bool
<code>GetRange(int i, int n)</code>	Return a sub-list of n elements from index i	List<T>
<code>Count</code>	Current element count (property)	int
<code>Capacity</code>	Current internal capacity (property)	int

Example 1.1 — Create and Basic Operations with List<int>

Create Program.cs in a new project, run each block and observe the output.

```

using System;
using System.Collections.Generic;

// — Create List and add elements —————
List<int> scores = new List<int>();

scores.Add(85);
scores.Add(92);
scores.Add(78);
scores.Add(95);
scores.Add(88);

Console.WriteLine($"Element count: {scores.Count}");
Console.WriteLine($"Capacity: {scores.Capacity}"); // always >= Count

// — Print the list —————
Console.Write("List: ");
foreach (int s in scores)
    Console.Write(s + " ");
Console.WriteLine();

// — Access by index —————
Console.WriteLine($"Element [0]: {scores[0]}");
Console.WriteLine($"Element [2]: {scores[2]}");

// — Modify an element —————
scores[1] = 99;
Console.WriteLine($"After setting [1] = 99: {scores[1]}");

```

OUTPUT

```

Element count: 5
Capacity: 8
List: 85 92 78 95 88
Element [0]: 85
Element [2]: 78
After setting [1] = 99: 99

```

 Note

Capacity is always $\geq$ Count. When Count exceeds Capacity, List automatically doubles its Capacity.

Pre-size with an estimate: `new List<int>(100)` avoids repeated resizing.

Example 1.2 — Insert, Remove, RemoveAll

```

List<string> fruits = new List<string> { "Apple", "Banana", "Cherry",
    "Date", "Elderberry" };

// — Insert at a specific position —————
fruits.Insert(2, "Blueberry"); // insert at index 2
Console.WriteLine("After Insert(2, Blueberry):");
fruits.ForEach(f => Console.Write(f + " "));
Console.WriteLine();

// — Remove by value —————
fruits.Remove("Date");
Console.WriteLine("\nAfter Remove(Date):");
fruits.ForEach(f => Console.Write(f + " "));

```

```

Console.WriteLine();

// — RemoveAt theo index —————
fruits.RemoveAt(0);
Console.WriteLine("\nAfter RemoveAt(0):");
fruits.ForEach(f => Console.Write(f + " "));
Console.WriteLine();

// — RemoveAll by condition —————
int removed = fruits.RemoveAll(f => f.Length > 6); // remove names longer
than 6 chars
Console.WriteLine($"After RemoveAll (Length > 6) - removed {removed}
element(s):");
fruits.ForEach(f => Console.Write(f + " "));
Console.WriteLine();

```

OUTPUT

```

After Insert(2, Blueberry):
Apple  Banana  Blueberry  Cherry  Date  Elderberry

After Remove(Date):
Apple  Banana  Blueberry  Cherry  Elderberry

After RemoveAt(0):
Banana  Blueberry  Cherry  Elderberry

After RemoveAll (Length > 6) - removed 3 element(s):
Cherry

```

Example 1.3 — Searching: Contains, Find, FindAll, IndexOf

```

List<int> numbers = new List<int> { 10, 25, 33, 42, 17, 58, 33, 71, 88, 33
};

// — Contains —————
Console.WriteLine($"Contains(42): {numbers.Contains(42)}"); // True
Console.WriteLine($"Contains(99): {numbers.Contains(99)}"); // False

// — IndexOf / LastIndexOf —————
Console.WriteLine($"IndexOf(33): {numbers.IndexOf(33)}"); // 2
(first)
Console.WriteLine($"LastIndexOf(33): {numbers.LastIndexOf(33)}"); // 9
(last)

// — Find: first element matching condition —————
int first = numbers.Find(n => n > 50);
Console.WriteLine($"Find(n > 50): {first}"); // 58

// — FindAll: all elements matching condition —————
List<int> big = numbers.FindAll(n => n > 50);
Console.WriteLine($"FindAll(n > 50): ");
big.ForEach(n => Console.Write(n + " "));
Console.WriteLine();

// — Exists & TrueForAll —————
Console.WriteLine($"Exists(n < 0): {numbers.Exists(n => n < 0)}");
// False
Console.WriteLine($"TrueForAll(n > 0): {numbers.TrueForAll(n => n > 0)}");
// True

```

OUTPUT

```
Contains(42): True
Contains(99): False
IndexOf(33): 2
LastIndexOf(33): 9
Find(n > 50): 58
FindAll(n > 50): 58 71 88
Exists(n < 0): False
TrueForAll(n > 0): True
```

Example 1.4 — Sorting: Sort, Reverse, Sort with Comparison

```
List<string> names = new List<string> { "Charlie", "Alice", "Bob", "Dave",
    "Eve" };

// — Default sort (alphabetical) —————
names.Sort();
Console.WriteLine("Sort(): ");
names.ForEach(n => Console.WriteLine(n + " "));
Console.WriteLine();

// — Reverse —————
names.Reverse();
Console.WriteLine("Reverse(): ");
names.ForEach(n => Console.WriteLine(n + " "));
Console.WriteLine();

// — Sort with Comparison — by name length —————
names.Sort((a, b) => a.Length.CompareTo(b.Length));
Console.WriteLine("Sort by length: ");
names.ForEach(n => Console.WriteLine(n + " "));
Console.WriteLine();

// — Sort complex objects —————
var students = new List<(string Name, int Score)>
{
    ("An", 85), ("Binh", 92), ("Chi", 78),
    ("Dung", 92), ("Em", 65)
};

// Sort: highest score first; same score → alphabetical by name
students.Sort((a, b) =>
{
    int cmp = b.Score.CompareTo(a.Score); // descending by score
    return cmp != 0 ? cmp : a.Name.CompareTo(b.Name); // ascending by name
});

Console.WriteLine("\nScore board (high → low):");
students.ForEach(s => Console.WriteLine($"{s.Name,-10} {s.Score}"));
```

OUTPUT

```
Sort(): Alice Bob Charlie Dave Eve
Reverse(): Eve Dave Charlie Bob Alice
Sort by length: Eve Bob Dave Alice Charlie

Score board (high → low):
    Binh      92
    Dung      92
    An        85
    Chi       78
```

Example 1.5 — GetRange, AddRange, ToArray, CopyTo

```

List<int> data = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

// — GetRange: extract a sub-list —————
List<int> mid = data.GetRange(3, 4);    // from index 3, take 4 elements
Console.WriteLine("GetRange(3, 4): ");
mid.ForEach(n => Console.Write(n + "  "));    // 4 5 6 7
Console.WriteLine();

// — AddRange: add multiple elements —————
List<int> extra = new List<int> { 11, 12, 13 };
data.AddRange(extra);
Console.WriteLine($"After AddRange: Count = {data.Count}");

// — ToArray —————
int[] arr = data.ToArray();
Console.WriteLine($"ToArray()[0] = {arr[0]},  arr.Length = {arr.Length}");

// — CopyTo —————
int[] dest = new int[5];
data.CopyTo(2, dest, 0, 5);    // copy 5 elements from index 2 into dest
Console.WriteLine("CopyTo (5 elements from index 2): ");
foreach (int x in dest) Console.Write(x + "  " );
Console.WriteLine();

```

OUTPUT

```

GetRange(3, 4): 4  5  6  7
After AddRange: Count = 13
ToArray()[0] = 1,  arr.Length = 13
CopyTo (5 elements from index 2): 3  4  5  6  7

```

1.3 ArrayList — Non-Generic List

ArrayList lives in the System.Collections namespace and is the predecessor of List<T>. It stores elements as object, meaning any type can be stored in the same ArrayList — but this is also its weakness: no type safety and casting is required when reading back.

⚠ ArrayList vs List<T>

ArrayList is no longer recommended for new code (.NET 2.0+).

Use List<T> instead whenever possible. ArrayList is still studied to understand legacy code.

Boxing/Unboxing: storing int in ArrayList costs extra memory and CPU time.

Method / Property	Description	Return
<code>Add(object value)</code>	Add one element to the end	<code>int (new index)</code>
<code>AddRange (ICollection)</code>	Add multiple elements	<code>void</code>

<code>Insert(int i, object)</code>	Insert at position i	void
<code>Remove(object value)</code>	Remove the first occurrence	void
<code>RemoveAt(int i)</code>	Remove element at index i	void
<code>Clear()</code>	Remove all elements	void
<code>Contains(object)</code>	Check if element exists	bool
<code>IndexOf(object)</code>	Find first index (-1 if not found)	int
<code>Sort()</code>	Sort (all elements must be the same type)	void
<code>Reverse()</code>	Reverse order	void
<code>ToArray()</code>	Convert to object[]	object[]
<code>ToArray(Type)</code>	Convert to a typed array	Array
<code>Count</code>	Current element count	int
<code>Capacity</code>	Internal capacity	int

Example 1.6 — Basics with ArrayList

```
using System.Collections;

ArrayList list = new ArrayList();

// Can store different types (but should not!)
list.Add(42);
list.Add("Hello");
list.Add(3.14);
list.Add(true);

Console.WriteLine($"Count: {list.Count}");

// Must cast when reading – easy source of runtime errors!
int num = (int)list[0];
string str = (string)list[1];
Console.WriteLine($"[0] = {num}, [1] = {str}");

// Practical ArrayList: store same type so Sort() works
ArrayList scores = new ArrayList { 55, 72, 88, 61, 94, 45 };
scores.Sort();
Console.Write("Sau Sort(): ");
foreach (object s in scores)
    Console.Write(s + " ");
Console.WriteLine();

// Convert to int[]
int[] arr = (int[])scores.ToArray(typeof(int));
Console.WriteLine($"ToArray – last element: {arr[arr.Length - 1]}");
```

OUTPUT

```
Count: 4
[0] = 42, [1] = Hello
Sau Sort(): 45 55 61 72 88 94
ToArray – last element: 94
```

1.4 LinkedList<T> — Doubly-Linked List

LinkedList<T> implements a doubly-linked list. Each element is a node containing: a value (Value), a pointer to the previous node (Previous), and a pointer to the next node (Next).

LinkedList<T> Memory Layout

`null ← [First: "A"] ↔ ["B"] ↔ ["C"] ↔ [Last: "D"] → null`

Each node is a `LinkedListNode<T>` with 3 properties:

`node.Value` — stored value

`node.Previous` — previous node (null if this is First)

`node.Next` — next node (null if this is Last)

Advantage: `AddFirst`, `AddLast`, `Remove` any specific node → $O(1)$

Disadvantage: No index access — must traverse sequentially

Method / Property	Description	Return
<code>AddFirst(T value)</code>	Add to the front of the list	<code>LinkedListNode<T></code>
<code>AddLast(T value)</code>	Add to the end of the list	<code>LinkedListNode<T></code>
<code>AddBefore(node, T value)</code>	Add before a specific node	<code>LinkedListNode<T></code>
<code>AddAfter(node, T value)</code>	Add after a specific node	<code>LinkedListNode<T></code>
<code>Remove(T value)</code>	Remove first occurrence of value	<code>bool</code>
<code>Remove(LinkedListNode<T>)</code>	Remove the specified node directly — $O(1)$	<code>void</code>
<code>RemoveFirst()</code>	Remove the first node	<code>void</code>
<code>RemoveLast()</code>	Remove the last node	<code>void</code>
<code>Find(T value)</code>	Find first node with value (from front)	<code>LinkedListNode<T></code>
<code>FindLast(T value)</code>	Find first node with value (from back)	<code>LinkedListNode<T></code>
<code>Contains(T value)</code>	Check if value exists in the list	<code>bool</code>
<code>Clear()</code>	Remove all elements	<code>void</code>
<code>CopyTo(T[], int)</code>	Copy to an array	<code>void</code>
<code>First</code>	First node (property)	<code>LinkedListNode<T></code>
<code>Last</code>	Last node (property)	<code>LinkedListNode<T></code>

Count	Element count	int
-------	---------------	-----

Example 1.7 — Basic Operations with LinkedList<T>

```
using System.Collections.Generic;

LinkedList<string> list = new LinkedList<string>();

// — Add elements —————
list.AddLast("B");
list.AddLast("D");
list.AddFirst("A");

// Find node "B" then insert "C" after it
LinkedListNode<string> nodeB = list.Find("B");
list.AddAfter(nodeB, "C");

// Add "E" to the end
list.AddLast("E");

PrintList(list, "Initial list");

// — Traverse in reverse direction —————
Console.Write("Reverse order: ");
LinkedListNode<string> cur = list.Last;
while (cur != null)
{
    Console.Write(cur.Value + " ");
    cur = cur.Previous;
}
Console.WriteLine();

// — Remove elements —————
list.Remove("C");
list.RemoveFirst();
PrintList(list, "After Remove(C) and RemoveFirst()");

// — Access First / Last —————
Console.WriteLine($"First: {list.First.Value}    Last: {list.Last.Value}");

// Helper method
static void PrintList(LinkedList<string> ll, string label)
{
    Console.Write($"{label,-40}: ");
    foreach (string item in ll) Console.Write(item + " ");
    Console.WriteLine($" (Count={ll.Count})");
}
```

OUTPUT	Initial list	: A B C D E
	(Count=5)	
	Reverse order:	E D C B A
	After Remove(C) and RemoveFirst()	: B D E (Count=3)
	First: B Last: E	

Example 1.8 — Real-World Use Case: Browser History with LinkedList<T>

The browser history bar (Back/Forward) is a classic LinkedList use case — moving left or right is $O(1)$.

```

LinkedList<string> history = new LinkedList<string>();
LinkedListNode<string> current = null;

void Navigate(string url)
{
    // Remove all pages forward of the current position
    while (current != null && current.Next != null)
        history.RemoveLast();

    var node = history.AddLast(url);
    current = node;
    Console.WriteLine($"[→] Navigate to: {current.Value}");
}

void Back()
{
    if (current?.Previous != null)
    {
        current = current.Previous;
        Console.WriteLine($"[←] Go back: {current.Value}");
    }
    else Console.WriteLine("[←] No previous page");
}

void Forward()
{
    if (current?.Next != null)
    {
        current = current.Next;
        Console.WriteLine($"[→] Go forward: {current.Value}");
    }
    else Console.WriteLine("[→] No next page");
}

// Simulate user actions
Navigate("google.com");
Navigate("youtube.com");
Navigate("github.com");
Back();
Back();
Forward();
Navigate("stackoverflow.com"); // new page → forward history cleared
Forward();                    // no next page

```

OUTPUT

```

[→] Navigate to: google.com
[→] Navigate to: youtube.com
[→] Navigate to: github.com
[←] Go back: youtube.com
[←] Go back: google.com
[→] Go forward: youtube.com
[→] Navigate to: stackoverflow.com
[→] No next page

```

PART 2 — Guided Lab Practice

Build a Student List Management App with interactive menu | ≈ 60 min

In this section you will build a complete Console application: a student list manager using `List<T>`. The application has an interactive menu so the user can choose any function at runtime.

Application Goals — Student Manager

- ① Add a new student
- ② Display all students
- ③ Search students by name
- ④ Find highest / lowest scoring student
- ⑤ Sort the list (by name or by score)
- ⑥ Delete a student
- ⑦ Filter students by score range
- ⑧ Statistics: average score, pass/fail counts
- 0 — Exit

STEP

Create the project and Student class

1

Open a terminal and run:

```
dotnet new console -n StudentManager --framework net8.0
cd StudentManager
code .
```

Create Student.cs:

```
// File: Student.cs
namespace StudentManager;

public class Student
{
    public string Id    { get; set; }
    public string Name  { get; set; }
    public double Score { get; set; }
    public string Major { get; set; }

    public Student(string id, string name, double score, string major)
    {
        Id    = id;
        Name  = name;
        Score = score;
        Major = major;
    }
}
```

```
// Academic grade classification
public string Grade => Score switch
{
    >= 9.0 => "Excellent",
    >= 8.0 => "Very Good",
    >= 7.0 => "Good",
    >= 5.0 => "Average",
    -      => "Weak",
};

public override string ToString()
    => $"{Id,-12} {Name,-20} {Score,5:F1} [{Grade}] {Major}";
}
```

STEP

Create StudentService class — all business logic**2**

Create StudentService.cs. This is the brain of the application — it processes the list using List<Student>.

```
// File: StudentService.cs
using System;
using System.Collections.Generic;

namespace StudentManager;

public class StudentService
{
    // — Student list stored in List<T> —————
    private List<Student> _students = new List<Student>();

    // — 1. Add student —————
    public void Add(Student s)
    {
        // Check for duplicate ID
        if (_students.Exists(x => x.Id == s.Id))
        {
            Console.WriteLine($" X ID {s.Id} already exists!");
            return;
        }
        _students.Add(s);
        Console.WriteLine($" ✓ Added: {s.Name}");
    }

    // — 2. Display all —————
    public void DisplayAll()
    {
        if (_students.Count == 0)
        {
            Console.WriteLine(" (List is empty)");
            return;
        }
        Console.WriteLine($" {"ID",-12} {"Full Name",-20} {"Score",5}
Grade      Major");
        Console.WriteLine(" " + new string('-', 68));
        _students.ForEach(s => Console.WriteLine(" " + s));
        Console.WriteLine($" Total: {_students.Count} student(s)");
    }
}
```

```

// — 3. Search by name —————
public void SearchByName(string keyword)
{
    // FindAll + lowercase keyword for case-insensitive search
    List<Student> found = _students.FindAll(
        s => s.Name.ToLower().Contains(keyword.ToLower()));

    if (found.Count == 0)
        Console.WriteLine($" No student found with keyword:
'{keyword}'");
    else
    {
        Console.WriteLine($" Found {found.Count} result(s):");
        found.ForEach(s => Console.WriteLine(" " + s));
    }
}

// — 4. Highest / lowest scoring student —————
public void ShowMinMax()
{
    if (_students.Count == 0) { Console.WriteLine(" (Empty)"); return;
}

    // Use ForEach to find min/max
    Student max = _students[0];
    Student min = _students[0];
    _students.ForEach(s => {
        if (s.Score > max.Score) max = s;
        if (s.Score < min.Score) min = s;
    });
    Console.WriteLine($" 🏆 Highest: {max}");
    Console.WriteLine($" 📉 Lowest: {min}");
}

// — 5. Sort —————
public void Sort(string by)
{
    if (by == "name")
        _students.Sort((a, b) => a.Name.CompareTo(b.Name));
    else // by score descending
        _students.Sort((a, b) => b.Score.CompareTo(a.Score));

    Console.WriteLine($" ✓ Sorted by {(by=="name"? "name": "score")}");
    DisplayAll();
}

// — 6. Delete by ID —————
public void DeleteById(string id)
{
    // Use RemoveAll with a lambda
    int count = _students.RemoveAll(s => s.Id == id);
    if (count > 0) Console.WriteLine($" ✓ Deleted student ID: {id}");
    else Console.WriteLine($" ✗ ID not found: {id}");
}

// — 7. Filter by score range —————
public void FilterByScore(double min, double max)
{
    List<Student> result = _students.FindAll(
        s => s.Score >= min && s.Score <= max);
}

```

```

        Console.WriteLine($" Students in score range [{min} - {max}]
({result.Count} result(s)):");
        if (result.Count == 0) Console.WriteLine(" (None)");
        else result.ForEach(s => Console.WriteLine(" " + s));
    }

    // — 8. Statistics —————
    public void Statistics()
    {
        if (_students.Count == 0) { Console.WriteLine(" (Empty)"); return;
    }

    double sum = 0;
    _students.ForEach(s => sum += s.Score);
    double avg = sum / _students.Count;

    int pass = _students.FindAll(s => s.Score >= 5.0).Count;
    int fail = _students.Count - pass;

    Console.WriteLine($" Total students : {_students.Count}");
    Console.WriteLine($" Average score : {avg:F2}");
    Console.WriteLine($" Pass (>= 5.0) : {pass}");
    Console.WriteLine($" Fail (< 5.0) : {fail}");

    // Count by grade using FindAll
    Console.WriteLine(" Classification:");
    foreach (string g in new[] {"Excellent", "Very
Good", "Good", "Average", "Weak"})
    {
        int cnt = _students.FindAll(s => s.Grade == g).Count;
        if (cnt > 0) Console.WriteLine($" {g,-12}: {cnt}");
    }
}

```

STEP

Build Program.cs — Interactive Menu**3**

This is the Main entry point with the main menu loop. Study how the menu is structured with a while loop.

```

// File: Program.cs
using System;
using StudentManager;

var service = new StudentService();

// — Sample data —————
service.Add(new Student("SE001", "Nguyen Van An", 8.5, "Software Eng"));
service.Add(new Student("SE002", "Tran Thi Binh", 7.2, "Software Eng"));
service.Add(new Student("IT001", "Le Van Chi", 9.1, "IT"));
service.Add(new Student("IT002", "Pham Thi Dung", 6.8, "IT"));
service.Add(new Student("SE003", "Hoang Van Em", 4.5, "Software Eng"));
service.Add(new Student("IT003", "Vo Thi Phuong", 8.9, "IT"));
Console.Clear();

// — Main menu loop —————
bool running = true;
while (running)
{

```

```

Console.WriteLine();
Console.WriteLine(" ");
Console.WriteLine("    STUDENT MANAGER – Collections ");
Console.WriteLine(" ");
Console.WriteLine(" 1. Add student ");
Console.WriteLine(" 2. Display all ");
Console.WriteLine(" 3. Search by name ");
Console.WriteLine(" 4. Highest / Lowest score ");
Console.WriteLine(" 5. Sort list ");
Console.WriteLine(" 6. Delete student ");
Console.WriteLine(" 7. Filter by score range ");
Console.WriteLine(" 8. Statistics ");
Console.WriteLine(" 0. Exit ");
Console.WriteLine(" ");
Console.Write(" Choose option: ");

string choice = Console.ReadLine()?.Trim() ?? "0";

Console.WriteLine();
switch (choice)
{
    case "1":
        Console.Write(" ID: "); string id = Console.ReadLine();
        Console.Write(" Name: "); string name = Console.ReadLine();
        Console.Write(" Score: "); double score =
double.Parse(Console.ReadLine());
        Console.Write(" Major: "); string major = Console.ReadLine();
        service.Add(new Student(id, name, score, major));
        break;

    case "2":
        service.DisplayAll();
        break;

    case "3":
        Console.Write(" Enter name keyword: ");
        service.SearchByName(Console.ReadLine());
        break;

    case "4":
        service.ShowMinMax();
        break;

    case "5":
        Console.Write(" Sort by (name/score): ");
        service.Sort(Console.ReadLine()?.ToLower() ?? "name");
        break;

    case "6":
        Console.Write(" Enter ID to delete: ");
        service.DeleteById(Console.ReadLine());
        break;

    case "7":
        Console.Write(" Score from: "); double lo =
double.Parse(Console.ReadLine());
        Console.Write(" Score to: "); double hi =
double.Parse(Console.ReadLine());
        service.FilterByScore(lo, hi);
        break;

    case "8":
        service.Statistics();

```

```

        break;

    case "0":
        Console.WriteLine("  Goodbye!");
        running = false;
        break;

    default:
        Console.WriteLine("  ✕ Invalid option.");
        break;
    }
}

```

STEP

Run and test each menu function**4**

Run dotnet run, then test each menu option. Sample output when selecting option 8 (Statistics):

OUTPUT

```

Total students   : 6
Average score    : 7.50
Pass (>= 5.0)    : 5
Fail (< 5.0)     : 1
Classification:
  Excellent      : 1
  Very Good      : 2
  Good           : 1
  Average        : 1
  Weak           : 1

```

 Testing Checklist

Try menu 5 (Sort by score), then menu 2 (Display) to see the order change.

Try menu 3, type "an" (lowercase) to see case-insensitive search in action.

Add a student with a duplicate ID to trigger the error message.

Try menu 7: score range 7.0–8.9 to filter students with Good and Very Good grades.

 Think About It

1. Why use List<Student> instead of a plain Student[] array?
2. RemoveAll returns an int — what does that value represent? Why is it useful?
3. If you need to find a student by ID, what method would you add to StudentService?
4. FindAll creates a brand-new List<Student> — is that good or bad? When should you be concerned?

PART 3 — Self-Practice Assignments — Submit for Grade

Complete and upload to Google Drive before the next session

Submission Instructions

How to Organise and Submit

Step 1: Go to the Google Drive link:

https://drive.google.com/drive/folders/1L_cT8Zi2EPv8tIXrTNn7-04tVWSq_uHv?usp=sharing

Step 2: Create your personal folder with the exact naming format:

StudentID_FullName

Examples: SE123456_NguyenVanAn

IT987654_TranThiBinh

Step 3: Inside your folder, create a sub-folder:

Lab02_Session2_Collections

Step 4: Upload your entire .NET project folder into it.

(Include: *.cs, *.csproj — do NOT upload bin/ and obj/ folders)

Step 5: Make sure the project runs cleanly: dotnet run with no errors.

Important Rules

Wrong folder name = assignment will NOT be graded.

No submission = 0 for this lab session.

Late submissions accepted up to 24 hours after class — maximum score is 70%.

Assignment 1 — Library Book Manager (30 points)

A1

**Library Book
Manager**

Build a library management application using List<Book>

★★★

Requirements

Create a new project named LibraryManager. Build a Book class and a menu-driven app with the structure below:

```
// Book class must have:
public class Book
{
    public string ISBN    { get; set; }    // book code
    public string Title   { get; set; }    // title
    public string Author  { get; set; }    // author
    public int    Year     { get; set; }    // publication year
    public double Price   { get; set; }    // price (thousands VND)
    public string Genre   { get; set; }    // genre: Tech / Fiction /
    Science / ...
}
```

Required Menu (minimum 7 functions):

1. Add a new book (validate: ISBN must be unique)
2. Display all books
3. Search books by author name (case-insensitive, partial match)
4. Sort by publication year (oldest first) or by price (lowest first)
5. Delete a book by ISBN
6. Filter books by genre
7. Statistics: total books, average price, most/least expensive, count per Genre

Minimum sample data: 8 books covering at least 3 different genres.

Technical Constraints

Use these methods: Add, FindAll, RemoveAll, Sort with Comparison, ForEach.

Do NOT use LINQ (Where, Select, OrderBy...) — only List<T> methods are allowed.

The menu must use a while/do-while loop so the user can make multiple selections.

Assignment 2 — Comparing List<T> vs ArrayList (35 points)

A2

Collection Comparison App

Build an app that demonstrates the differences between List<T> and ArrayList

★★★

Requirements — 3 Sections:

Section 2A — Type Safety Demo

Implement menu option "Type Safety Test" that:

8. Creates an ArrayList and adds: 5 int numbers, 3 strings, 2 doubles into the same list.
9. Prints all elements using foreach — no casting needed for printing (Console.WriteLine(obj) works).
10. Attempts a wrong cast: assigns a string element to an int variable — catches InvalidCastException and prints a message.

- Creates a `List<int>` with the same numeric data — shows the compiler catches `Add("hello")` at compile time.

Section 2B — Performance Benchmark

Implement menu option "Performance Test" that measures time with Stopwatch:

- Adds 1,000,000 int elements to both an `ArrayList` and a `List<int>` — records time for each.
- Reads all 1,000,000 elements (with casting for `ArrayList`) — records time.
- Prints a clear comparison table of results.

Section 2C — Practical ArrayList

Implement menu option "Mixed Storage" that:

- Uses `ArrayList` to store log entries as objects — each entry is a tuple: (DateTime, string level, string message).
- Prints the 10 most recent log entries (end of list).
- Removes all entries where `level == "DEBUG"`.

Note

Each test must produce clear printed output — do not run silently.

The benchmark section must print time in ms and a conclusion (how many % faster is `List<T>`).

Assignment 3 — Real-World App with `LinkedList<T>` (35 points)

A3

Music Playlist Manager

Simulate a music player using `LinkedList<T>`

★★★

Build a music playlist management app. `LinkedList<T>` is ideal because songs are frequently inserted or removed at any position, and Prev/Next navigation must be $O(1)$.

Data class:

```
public class Song
{
    public string Title    { get; set; }
    public string Artist   { get; set; }
    public int    Duration { get; set; } // seconds
    public string Genre    { get; set; }

    public string DurationStr => $"{Duration/60}:{Duration%60:D2}";
    public override string ToString()
        => $"{Title,-30} {Artist,-20} {DurationStr} [{Genre}]";
}
```

Required Menu (all 9 functions):

- Add song to the end of the playlist (`AddLast`)
- Add song to the front of the playlist (`AddFirst`)
- Insert song after the currently playing song (`AddAfter` current node)

21. Remove currently playing song (Remove current node, move to next)
22. Next song (current = current.Next; if null → wrap to First)
23. Previous song (current = current.Previous; if null → wrap to Last)
24. Display full playlist (currently playing song marked with ►)
25. Search and jump to a song by title (Find → set current)
26. Statistics: total songs, total duration, count per Genre

Additional Requirements:

- The initial playlist must contain at least 6 pre-loaded songs.
- Next must support circular wrap: when on the last song, Next → goes back to First.
- Previous similarly: when on the first song, Prev → jumps to Last.
- Clearly display which song is currently playing (current node).

Technical Tips

Use `LinkedList<Song>` and keep a `LinkedListNode<Song>` current variable to track the playing song.

Circular next: if (`current.Next == null`) `current = list.First`; else `current = current.Next`;

When removing current: save next first, then Remove, then set `current = saved next`.

Use `Find()` to locate a node by title, then set `current = found node`.

Grading Rubric

Criterion	A1 (30 pts)	A2 (35 pts)	A3 (35 pts)
Program runs without errors	8 pts	8 pts	8 pts
All required menu functions present	10 pts	12 pts	12 pts
Correct use of List/ArrayList/LinkedList methods	7 pts	10 pts	10 pts
Clear, well-formatted output	3 pts	3 pts	3 pts
Clean code with comments	2 pts	2 pts	2 pts

Scoring

Submit all 3 assignments: maximum 100 points.

Submit 2 assignments: max 65 pts (A1+A2) or 70 pts (A1+A3).

Submit 1 assignment: maximum 35 points.

Copied code: both students receive 0.